

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente “CUNOC”
División Ciencias de la Ingeniería
Introducción a la Programación y Computación IPC 2
Ing. Moisés Granados Guevara



Manual Técnico

Sistema Web para la Administración de Buses

William Eduardo Gómez Sánchez
Carné: 202232064

Guatemala, Quetzaltenango 17 de septiembre de 2026

Introducción

El presente manual técnico documenta el diseño e implementación del sistema CodeNBuses, una aplicación web desarrollada para la empresa de desarrollo de software "Code 'n Bugs", cuyo propósito es brindar a una empresa de transporte extraurbano de Guatemala una plataforma integral para la gestión de su flota de buses, sus operaciones de viaje, y sus finanzas.

El sistema fue desarrollado utilizando el lenguaje de programación Java, empleando JavaServer Pages (JSP) como tecnología de presentación del lado del servidor, en conjunto con HTML, CSS y JavaScript para la construcción de la interfaz de usuario. La persistencia de la información se gestiona mediante una base de datos relacional MySQL, a la cual la aplicación se conecta utilizando JDBC (Java Database Connectivity).

Este documento está dirigido a desarrolladores, personal técnico, o cualquier persona interesada en comprender la arquitectura, el diseño de clases, la estructura de la base de datos, y el funcionamiento interno del sistema. Se detallan los requisitos necesarios para su instalación y ejecución, la arquitectura empleada bajo el patrón de diseño Modelo-Vista-Controlador (MVC), la descripción de las clases del sistema organizadas por paquete, los diagramas de análisis y diseño correspondientes, así como el pseudocódigo de los procesos más relevantes de la aplicación.

Requisitos del Sistema

Requisitos Mínimos para Ejecutar el Proyecto

- Sistema operativo Windows 10 o superior
- Java Development Kit (JDK) versión 17 o superior
- Apache Tomcat 10.1.59 o compatible con Jakarta EE 10
- MySQL Server 8.0 o superior
- MySQL Workbench (opcional, recomendado para administrar la base de datos)
- Navegador web actualizado (Google Chrome, Microsoft Edge, o similar)
- Mínimo 4 GB de memoria RAM disponible
- Conexión a internet (necesaria para cargar las librerías de Bootstrap, Bootstrap Icons y Leaflet desde su CDN)

Requisitos para Compilar el Proyecto

- Apache NetBeans (última versión estable)
- Apache Maven (integrado en NetBeans, utilizado para la gestión de dependencias del proyecto)
- Las siguientes dependencias, declaradas en el archivo `pom.xml`:
 - `jakarta.jakartaee-api` versión 10.0.0
 - `mysql-connector-j` versión 8.3.0

Compilar y Ejecutar desde NetBeans

1. Abrir Apache NetBeans e importar el proyecto mediante la opción *Open Project*.
2. Verificar que el servidor Apache Tomcat esté correctamente configurado en NetBeans (*Tools* → *Servers*).
3. Asegurarse de que la base de datos `gestion_buses` exista en MySQL, ejecutando previamente el script SQL correspondiente.
4. Dar clic derecho sobre el proyecto y seleccionar *Clean and Build*, para compilar el proyecto y descargar las dependencias necesarias.
5. Dar clic derecho sobre el proyecto y seleccionar *Run*, lo cual desplegará automáticamente la aplicación en el servidor Tomcat configurado.
6. Acceder a la aplicación desde el navegador en la dirección <http://localhost:8080/ProyectoCodeNBuses/>.

Compilar desde Consola (Maven)

1. Abrir una terminal en la carpeta raíz del proyecto (donde se encuentra el archivo `pom.xml`).
2. Ejecutar el comando `mvn clean package` para compilar el proyecto y generar el archivo `.war` correspondiente en la carpeta `target/`.

3. Copiar el archivo `.war` generado a la carpeta `webapps` de la instalación de Apache Tomcat.
4. Iniciar el servidor Tomcat ejecutando el script `startup.bat` (Windows) ubicado en la carpeta `bin` de la instalación de Tomcat.
5. Acceder a la aplicación desde el navegador en la dirección correspondiente.

Descripción de Tecnologías

Java (JDK 17+)

Lenguaje de programación principal utilizado para el desarrollo de la lógica de negocio del sistema, tanto en la capa de controladores (Servlets) como en la capa de acceso a datos (persistencia).

Jakarta Server Pages (JSP) — Jakarta EE 10

Tecnología utilizada para la generación dinámica de contenido HTML del lado del servidor. Permite combinar código HTML estático con fragmentos de código Java (scriptlets) para construir las vistas de la aplicación, mostrando información dinámica proveniente de la base de datos.

Apache Tomcat 10.1.59

Contenedor de servlets (servidor de aplicaciones) utilizado para desplegar y ejecutar la aplicación web, procesando las peticiones HTTP de los usuarios y sirviendo las páginas JSP generadas.

MySQL 8.0

Sistema gestor de bases de datos relacional utilizado para el almacenamiento persistente de toda la información del sistema (usuarios, buses, viajes, boletos, transacciones financieras, entre otros).

JDBC (Java Database Connectivity)

API utilizada para establecer la comunicación entre la aplicación Java y la base de datos MySQL, permitiendo la ejecución de consultas SQL mediante sentencias preparadas (`PreparedStatement`), lo cual protege al sistema contra ataques de inyección SQL.

Bootstrap 5.3

Framework de CSS utilizado para el diseño responsivo de la interfaz de usuario, proporcionando componentes visuales predefinidos como botones, formularios, tablas, y ventanas modales.

Bootstrap Icons

Biblioteca de iconografía vectorial utilizada para complementar visualmente la interfaz, representando de forma gráfica las distintas acciones y roles del sistema.

JavaScript

Lenguaje utilizado del lado del cliente para implementar interactividad en la interfaz, principalmente en la gestión de las ventanas modales de confirmación personalizadas.

Leaflet

Biblioteca de JavaScript de código abierto utilizada para la generación del mapa interactivo de rutas, empleando los datos cartográficos de OpenStreetMap.

Apache Maven

Herramienta de gestión y automatización utilizada para la administración de las dependencias del proyecto y su proceso de compilación.

Git

Sistema de control de versiones utilizado a lo largo de todo el desarrollo del proyecto para el seguimiento de los cambios realizados en el código fuente.

Conceptos de Programación Orientada a Objetos Aplicados

Herencia

El sistema aplica herencia de forma central en la jerarquía de usuarios y de viajes:

- La clase **Usuario** actúa como clase padre, de la cual heredan **AdminSistema**, **AdminSucursal**, **Chofer**, y **ClienteRegular**. Todas comparten los atributos comunes de un usuario (nombre, NIT, DPI, teléfono, dirección, correo, contraseña), mientras que cada subclase agrega sus propios atributos y comportamientos específicos (por ejemplo, **Chofer** agrega número de licencia y salario base).
- La clase abstracta **Viaje** actúa como clase padre de **ViajeRegular** y **ViajePrivado**, compartiendo los atributos comunes de cualquier viaje (bus, chofer, fechas), mientras cada subclase implementa su propia lógica específica.

Clases Abstractas y Polimorfismo

La clase **Viaje** es abstracta y declara el método **calcularSalarioChofer()** también como abstracto. Cada subclase (**ViajeRegular** y **ViajePrivado**) sobrescribe este método (**@Override**) con su propia implementación: **ViajeRegular** retorna el salario

base del chofer, mientras que `ViajePrivado` retorna el salario base incrementado en un 15%. Esto permite tratar de forma uniforme a cualquier tipo de viaje al momento de calcular el salario correspondiente, sin necesidad de conocer su tipo específico en tiempo de ejecución.

Encapsulamiento

Todos los atributos de las clases del sistema se declaran como `private`, exponiendo su acceso únicamente a través de métodos públicos `get` y `set` (getters y setters). Esto protege la integridad de los datos, evitando que sean modificados directamente desde fuera de la clase sin pasar por la validación correspondiente.

Interfaces

Se define la interfaz genérica `Persistencia<T>`, la cual establece un contrato común (`insertar`, `actualizar`, `eliminar`, `buscarPorId`, `listarTodos`) que deben implementar todas las clases de persistencia del sistema (`BusPersistencia`, `ChoferPersistencia`, `RutaPersistencia`, entre otras). Esto garantiza consistencia en el manejo de operaciones de base de datos a lo largo de todo el proyecto.

Manejo de Excepciones Personalizadas

El sistema define un conjunto de excepciones propias (heredando de una clase base `ExcepcionCodeNBuses`), como `ExcepcionCreditoInsuficiente`, `ExcepcionAsientoNoDisponible`, `ExcepcionViajeNoEliminable`, entre otras. Estas excepciones permiten manejar de forma controlada y específica los distintos errores de negocio que pueden ocurrir durante la operación del sistema, mostrando mensajes claros al usuario en lugar de exponer errores técnicos.

Patrones de Diseño

Modelo-Vista-Controlador (MVC)

El sistema está estructurado bajo el patrón arquitectónico MVC, separando claramente las responsabilidades en tres capas:

- **Modelo:** representado por las clases de entidad (paquete `entidad`) y las clases de persistencia (paquete `persistencia`), encargadas de representar los datos del negocio y gestionar su almacenamiento en la base de datos.
- **Vista:** representada por los archivos JSP (ubicados en `src/main/webapp/vistas`), encargados de mostrar la información al usuario y recibir sus interacciones a través de formularios.

- **Controlador:** representado por las clases Servlet (paquete `controlador`), encargadas de recibir las peticiones HTTP del usuario, coordinar la lógica de negocio correspondiente, y determinar qué vista debe mostrarse como respuesta.

Data Access Object (DAO)

Cada clase de persistencia (por ejemplo, `BusPersistencia`, `ChoferPersistencia`) actúa como un objeto de acceso a datos, encapsulando toda la lógica de comunicación con la base de datos para una entidad específica. Esto separa la lógica de acceso a datos de la lógica de negocio contenida en los controladores.

Despachador (Dispatcher)

La clase `ViajePersistencia` implementa un patrón de despachador: al buscar un viaje por su identificador, consulta la columna `tipo` de la base de datos y delega la construcción del objeto correspondiente hacia `ViajeRegularPersistencia` o `ViajePrivadoPersistencia`, según corresponda. De esta manera, el resto del sistema puede trabajar con la clase abstracta `Viaje` sin necesidad de conocer su tipo específico.

Transacciones

En las operaciones que involucran la modificación de más de una tabla de forma simultánea (por ejemplo, el registro de un nuevo chofer, que afecta tanto la tabla `usuario` como `chofer`), se utiliza el manejo explícito de transacciones mediante `Connection.setAutoCommit(false)`, garantizando que ambas operaciones se completen exitosamente juntas, o que ninguna de ellas se aplique en caso de error (`rollback`).

Características Técnicas

- **Validación de datos en el lado del servidor:** todas las entradas de usuario son validadas antes de ser procesadas o almacenadas, verificando formato, obligatoriedad, y reglas de negocio específicas (por ejemplo, el DPI debe contener exactamente 13 dígitos numéricos).
- **Prevención de inyección SQL:** todas las consultas a la base de datos se realizan mediante sentencias preparadas (`PreparedStatement`), evitando la concatenación directa de valores ingresados por el usuario dentro de las consultas SQL.
- **Control de acceso basado en roles:** el sistema verifica, en cada Controlador, que el usuario autenticado tenga el rol adecuado para ejecutar la acción solicitada, utilizando el operador `instanceof` sobre la jerarquía de herencia de `Usuario`.

- **Registros inmutables:** los registros de salida y llegada de los viajes, una vez guardados, no pueden ser modificados ni eliminados, garantizando la integridad de la información operativa del sistema, conforme a lo establecido en el enunciado.
- **Subida de archivos:** el sistema permite la carga de fotografías para buses y choferes mediante formularios de tipo `multipart/form-data`, validando que el archivo subido corresponda a un formato de imagen válido (JPG, JPEG, PNG, o GIF) antes de almacenarlo en el servidor.
- **Interfaz responsiva:** la interfaz de usuario se adapta a distintos tamaños de pantalla mediante el uso del framework Bootstrap, garantizando una experiencia de uso adecuada en distintos dispositivos.
- **Exportación de reportes:** los nueve reportes del sistema permiten ser exportados en formato HTML, respetando los filtros aplicados por el usuario al momento de la exportación.
- **Mapa interactivo:** el reporte de mapa de rutas utiliza la biblioteca Leaflet para representar gráficamente las rutas de una sucursal específica sobre un mapa real, mostrando marcadores en las ubicaciones de las sucursales conectadas.

Arquitectura del Proyecto

Patrón Modelo-Vista-Controlador (MVC)

El sistema CodeNBuses está organizado siguiendo el patrón de arquitectura Modelo-Vista-Controlador, el cual separa la aplicación en tres capas con responsabilidades claramente diferenciadas, facilitando el mantenimiento, la escalabilidad y la comprensión del código.

El flujo de una petición dentro del sistema sigue el siguiente recorrido:

1. El usuario interactúa con una **Vista** (archivo JSP), ya sea navegando un enlace o enviando un formulario.
2. La petición HTTP es recibida por el **Controlador** correspondiente (Servlet), el cual determina la acción solicitada mediante el parámetro `accion`.
3. El Controlador invoca a la capa de **Persistencia**, la cual se comunica con la base de datos MySQL para obtener o almacenar la información necesaria, construyendo objetos del **Modelo** (entidades).
4. El Controlador coloca la información obtenida en el objeto `request`, y realiza un `forward` o `redirect` hacia la Vista correspondiente.
5. La Vista renderiza el HTML final, incorporando la información dinámica proporcionada por el Controlador, y lo envía de vuelta al navegador del usuario.

Esta separación permite que cada capa evolucione de forma independiente: la interfaz visual puede modificarse sin afectar la lógica de negocio, y la lógica de acceso a datos puede optimizarse sin afectar la presentación.

Estructura de Paquetes

El proyecto se organiza en los siguientes paquetes principales, dentro del paquete raíz `com.usac.buses.proyectocodenbuses`:

- **entidad**: contiene las clases que representan los objetos de negocio del sistema (Usuario y sus subclases, Bus, Sucursal, Viaje y sus subclases, Boleto, Cartera, entre otras), así como las enumeraciones utilizadas (`EstadoOperativo`, `TipoLicencia`, `TipoMovimiento`).
- **excepcion**: contiene las clases de excepciones personalizadas del sistema, utilizadas para representar errores específicos del negocio.
- **persistencia**: contiene las clases encargadas de la comunicación con la base de datos, implementando la interfaz genérica `Persistencia<T>` para las operaciones estándar de creación, lectura, actualización y eliminación (o desactivación, según corresponda).
- **controlador**: contiene las clases Servlet que reciben las peticiones HTTP, coordinan la lógica de negocio, y determinan la vista de respuesta.
- **utilidades**: contiene clases auxiliares del sistema, como constantes utilizadas en distintos cálculos (por ejemplo, el porcentaje adicional de salario para viajes privados).

Adicionalmente, dentro de la carpeta `src/main/webapp` se encuentra la carpeta `vistas`, organizada en subcarpetas por módulo (bus, chofer, ruta, viaje, boleto, cartera, gasto, admin, sucursal, reporte, usuario), conteniendo los archivos JSP correspondientes a cada funcionalidad del sistema, así como la carpeta `comunes` con los elementos compartidos entre vistas (encabezado, pie de página, y exportación HTML), y la carpeta `recursos`, que contiene los archivos CSS del sistema y las imágenes subidas por los usuarios.

Organización de las Vistas (JSP)

A diferencia de las clases Java, que se organizan mediante el sistema de paquetes, los archivos JSP encargados de la presentación se organizan dentro de la carpeta `src/main/webapp/vistas`, siguiendo una estructura de carpetas por módulo funcional del sistema:

- **usuario**: contiene las vistas relacionadas con el acceso y perfil del usuario (inicio de sesión, creación de cuenta, perfil, edición de perfil).
- **bus**: contiene las vistas para la gestión de buses (listado, registro, edición).

- **chofer**: contiene las vistas para la gestión de choferes.
- **ruta**: contiene las vistas para la gestión de rutas.
- **viaje**: contiene las vistas para la gestión de viajes regulares, incluyendo el registro de salida y llegada.
- **alquiler**: contiene las vistas relacionadas con la solicitud y confirmación de alquileres privados.
- **boleto**: contiene las vistas para la búsqueda de viajes disponibles, selección de asiento, y consulta de boletos comprados.
- **cartera**: contiene las vistas de la cartera digital del usuario (saldo, recarga, historial de movimientos).
- **gasto**: contiene las vistas para el registro de gastos de taller.
- **admin**: contiene las vistas de administración del sistema (gestión de administradores de sucursal, configuración de depreciación).
- **sucursal**: contiene las vistas para la gestión de sucursales.
- **reporte**: contiene las vistas correspondientes a los nueve reportes del sistema.
- **comunes**: contiene los elementos compartidos entre todas las vistas del sistema: el encabezado (**header.jsp**, incluyendo la barra de navegación), el pie de página (**footer.jsp**), y el fragmento reutilizable para la exportación de reportes en formato HTML (**exportarHTML.jsp**).

Adicionalmente, la carpeta **recursos** contiene los archivos estáticos del sistema, organizados en **css** (estilos personalizados del sistema) y una carpeta **imagenes**, generada dinámicamente por el sistema al momento de que un usuario sube una fotografía de bus o chofer.

Descripción de Clases por Paquete

Paquete **entidad**

Usuario (clase padre)

Representa a cualquier persona registrada en el sistema. Contiene los atributos comunes a todos los tipos de usuario: **idUsuario**, **nombre**, **nit**, **dpi**, **telefono**, **direccion**, **correo**, **contrasena**, **activo**. Incluye el método **editarPerfil()**, que permite actualizar los datos personales del usuario.

AdminSistema (hereda de Usuario)

Representa al administrador con el mayor nivel de control sobre el sistema. Incluye el método **configurarDepreciacion()**.

AdminSucursal (hereda de Usuario)

Representa al administrador responsable de una sucursal específica. Agrega el atributo **sucursal**, que indica a cuál sucursal pertenece.

Chofer (hereda de Usuario)

Representa a un conductor de la empresa. Agrega los atributos **numeroLicencia**, **tipoLicencia**, **fechaVencimiento**, **salarioBase**, **sucursal**, y **foto**.

ClienteRegular (hereda de Usuario)

Representa a cualquier persona que utiliza los servicios de transporte de la empresa como consumidor final.

Viaje (clase abstracta)

Representa un viaje realizado por un bus de la empresa. Contiene los atributos comunes: **idViaje**, **bus**, **chofer**, **fechaHoraSalida**, **fechaHoraLlegadaEstimada**. Declara el método abstracto **calcularSalarioChofer()**.

ViajeRegular (hereda de Viaje)

Representa un viaje programado entre dos sucursales, destinado a la venta de boletos al público. Agrega el atributo **ruta**. Implementa **calcularSalarioChofer()** retornando el salario base del chofer.

ViajePrivado (hereda de Viaje)

Representa un viaje de alquiler contratado por un cliente para un grupo específico de pasajeros, con origen y destino libres. Agrega los atributos **origen**, **destino**, **pasajeros**, **precioEstimado**, **precioConfirmado**, y **solicitante**. Implementa **calcularSalarioChofer()** retornando el salario base incrementado en un 15%.

Bus

Representa un bus de la flota de la empresa. Contiene los atributos **idBus**, **sucursal**, **placa**, **marca**, **modelo**, **anio**, **capacidad**, **foto**, **estadoOperativo**, **kilometraje**, y **activo**.

Sucursal

Representa una sucursal de la empresa. Contiene los atributos **idSucursal**, **nombre**, **ubicacion**, **latitud**, y **longitud** (estos dos últimos utilizados para la generación del mapa de rutas).

Ruta

Representa una ruta de viaje entre dos sucursales. Contiene los atributos **idRuta**, **sucursalOrigen**, **sucursalDestino**, **distanciaKm**, y **precioBoleto**.

Boleto

Representa la compra de un boleto por parte de un cliente. Contiene los atributos **idBoleto**, **viaje**, **cliente**, **numeroAsiento**, **fechaPago**, y **precio**.

Cartera

Representa la cartera digital de un usuario. Contiene los atributos **idCartera**, **usuario**, y **saldo**.

MovimientoCartera

Representa un movimiento (recarga o pago) realizado sobre una cartera. Contiene los atributos **idMovimiento**, **cartera**, **tipo**, **monto**, y **fecha**.

RegistroSalida

Representa el registro inmutable de la salida real de un viaje. Contiene los atributos **idRegistroSalida**, **viaje**, **horaSalidaReal**, y **kilometrajeSalida**.

RegistroLlegada

Representa el registro inmutable de la llegada real de un viaje. Contiene los atributos **idRegistroLlegada**, **viaje**, **horaLlegadaReal**, **kilometrajeLlegada**, **gastoCombustible**, y **depreciacionCalculada**. Calcula automáticamente la depreciación en base a los kilómetros recorridos en el viaje y el monto de depreciación vigente.

Gasto

Representa un gasto de taller o repuestos asociado a un bus. Contiene los atributos **idGasto**, **bus**, **montoManoObra**, **montoRepuestos**, y **fecha**.

Paquete **excepcion**

ExcepcionCodeNBuses (clase padre)

Clase base de la cual heredan todas las excepciones personalizadas del sistema.

ExcepcionCreditoInsuficiente

Se lanza cuando un usuario intenta realizar una compra (boleto o alquiler) sin contar con saldo suficiente en su cartera digital.

ExcepcionAsientoNoDisponible

Se lanza cuando un cliente intenta comprar un boleto para un asiento que ya se encuentra ocupado en el viaje seleccionado.

ExcepcionViajeNoEliminable

Se lanza cuando se intenta eliminar un viaje que ya ha sido iniciado (tiene registro de salida) o que ya cuenta con boletos vendidos.

ExcepcionTipoViajeInmutable

Se lanza cuando se intenta modificar el tipo de un viaje ya existente (regular o privado), lo cual no está permitido según el enunciado del sistema.

ExcepcionBusNoDesactivable

Se lanza cuando se intenta desactivar un bus que cuenta con viajes programados o en tránsito.

ExcepcionRutaNoEliminable

Se lanza cuando se intenta eliminar una ruta que aún cuenta con viajes asociados.

ExcepcionLicenciaNoValidaParaBus

Se lanza cuando se intenta registrar un chofer con un tipo de licencia distinto a A o B, las cuales son las únicas autorizadas para conducir transporte extraurbano de pasajeros en Guatemala.

ExcepcionFormatoInvalido

Se lanza cuando un dato ingresado por el usuario no cumple con el formato esperado (por ejemplo, un DPI que no contiene exactamente 13 dígitos numéricos). Contiene el atributo adicional **campo**, que indica el nombre del campo que generó el error.

Paquete **persistencia**

Persistencia<T> (interfaz genérica)

Define el contrato común que deben implementar las clases de persistencia del sistema:

insertar(T), **actualizar(T)**, **eliminar(int)**, **buscarPorId(int)**:

Optional<T>, y **listarTodos(): ArrayList<T>**. El uso de **Optional<T>**

permite manejar de forma explícita el caso en que un registro no sea encontrado, evitando el uso de valores nulos sin control.

ConexionBase

Clase encargada de establecer y cerrar la conexión con la base de datos MySQL mediante JDBC.

UsuarioPersistencia

Gestiona operaciones generales sobre la tabla **usuario**, comunes a todos los tipos de usuario. Destaca el método **autenticar(correo, contrasena)**, el cual identifica el tipo de usuario mediante la columna **tipo** de la base de datos, y despacha la construcción del objeto correspondiente hacia la clase de persistencia específica

(**AdminSistemaPersistencia**, **AdminSucursalPersistencia**, **ChoferPersistencia**, o **ClienteRegularPersistencia**), garantizando que el objeto retornado sea una instancia concreta del tipo de usuario correcto, necesaria para el correcto funcionamiento del control de acceso basado en roles.

AdminSistemaPersistencia, AdminSucursalPersistencia, ChoferPersistencia, ClienteRegularPersistencia

Implementan las operaciones específicas de creación, actualización y consulta para cada subtipo de usuario, incluyendo las columnas adicionales propias de cada uno. Las operaciones de inserción y actualización de **Chofer** y **AdminSucursal** utilizan transacciones, dado que afectan simultáneamente la tabla **usuario** y su tabla específica correspondiente.

BusPersistencia

Gestiona las operaciones sobre la tabla **bus**. Incluye el método **existeBusConPlaca(placa)**, utilizado para validar que no se registren buses con placas duplicadas.

SucursalPersistencia

Gestiona las operaciones sobre la tabla **sucursal**. Incluye el método **existeSucursalConNombre(nombre)**, utilizado para validar que no existan sucursales con nombres duplicados.

RutaPersistencia

Gestiona las operaciones sobre la tabla **ruta**, incluyendo métodos de consulta filtrados por sucursal.

ViajePersistencia

Actúa como despachador para la jerarquía de viajes. Su método **buscarPorId()** consulta la columna **tipo** de la tabla **viaje**, y delega la construcción del objeto hacia **ViajeRegularPersistencia** o **ViajePrivadoPersistencia** según corresponda, permitiendo trabajar de forma genérica con la clase abstracta **Viaje**.

ViajeRegularPersistencia

Gestiona las operaciones específicas de los viajes regulares. Destacan los métodos **choferTieneViajeEnHorario()** y **busTieneViajeEnHorario()**, utilizados para validar que un chofer o bus no sean asignados a dos viajes con horarios traslapados, y **choferTieneViajeActivo()** / **busTieneViajeActivo()**, utilizados para impedir la desactivación de un chofer o bus con viajes programados o en tránsito. Estos métodos consultan directamente la tabla padre **viaje**, por lo que cubren tanto viajes regulares como privados.

ViajePrivadoPersistencia

Gestiona las operaciones específicas de los viajes de alquiler privado, incluyendo el cálculo de ingresos por sucursal y fecha, y el registro del solicitante del alquiler.

BoletoPersistencia

Gestiona las operaciones sobre la tabla **boleto**. Incluye el método **comprarBoleto()**, el cual ejecuta dentro de una transacción el registro del boleto, la actualización del saldo de la cartera, y el registro del movimiento correspondiente. Incluye también **contarAsientosDisponibles()**, utilizado para calcular en tiempo real la disponibilidad de asientos de un viaje.

CarteraPersistencia y MovimientoCarteraPersistencia

Gestionan las operaciones relacionadas con la cartera digital de los usuarios y su historial de movimientos.

RegistroSalidaPersistencia y RegistroLlegadaPersistencia

Gestionan el registro de los eventos de salida y llegada de los viajes. El método **registrarLlegada()** de **RegistroLlegadaPersistencia** ejecuta dentro de una transacción el registro del evento y la actualización del kilometraje total del bus.

GastoPersistencia

Gestiona el registro de los gastos de taller y repuestos asociados a cada bus.

ConfiguracionPersistencia

Gestiona el monto de depreciación por kilómetro configurado por el Administrador del Sistema. El método **actualizarMontoDepreciacion()** inserta un nuevo registro en lugar de actualizar el existente, con el propósito de no afectar los cálculos de depreciación ya realizados con montos anteriores.

Paquete controlador

ControladorUsuario (/usuario)

Gestiona el inicio de sesión, la creación de cuentas de cliente, la visualización y edición del perfil de usuario, y el cierre de sesión.

ControladorSucursal (/sucursal)

Gestiona el registro, edición y listado de sucursales, exclusivo del rol Administrador del Sistema. Incluye la validación de nombre duplicado.

ControladorAdmin (/admin)

Gestiona el registro, edición, activación y desactivación de administradores de sucursal, así

como la configuración del monto de depreciación por kilómetro. Ambas funciones exclusivas del rol Administrador del Sistema.

ControladorBus (/bus)

Gestiona el registro, edición, desactivación y reactivación de buses, incluyendo la subida de fotografías mediante formularios multipart. Valida que un bus no pueda desactivarse si cuenta con viajes programados o en tránsito.

ControladorChofer (/chofer)

Gestiona el registro, edición, desactivación y reactivación de choferes, incluyendo la subida de fotografías y la validación de que el tipo de licencia sea A o B.

ControladorRuta (/ruta)

Gestiona el registro, edición y eliminación de rutas, validando que la sucursal de origen sea distinta a la de destino, y que una ruta no pueda eliminarse si tiene viajes asociados.

ControladorViaje (/viaje)

Gestiona el registro, edición y eliminación de viajes regulares, así como el registro de salida y llegada, funcionalidad compartida también con los viajes de alquiler privado. Valida la disponibilidad de horario del bus y chofer asignados, y verifica que solo el Administrador de Sucursal o el chofer específicamente asignado puedan registrar los eventos de salida y llegada de un viaje.

ControladorAlquiler (/alquiler)

Gestiona la solicitud de alquileres privados, el cálculo del precio estimado, y la confirmación del precio con la asignación simultánea de bus y chofer, aplicando las mismas validaciones de disponibilidad de horario utilizadas en los viajes regulares.

ControladorBoleto (/boleto)

Gestiona la búsqueda de viajes disponibles, la selección de asiento, y la compra de boletos, validando la disponibilidad del asiento, el saldo suficiente en la cartera del cliente, y que el viaje no haya pasado ya su fecha de salida.

ControladorCartera (/cartera)

Gestiona la consulta del saldo, la recarga manual de la cartera digital, y el historial de movimientos del usuario.

ControladorGasto (/gasto)

Gestiona el registro y consulta de gastos de taller y repuestos, filtrados por bus.

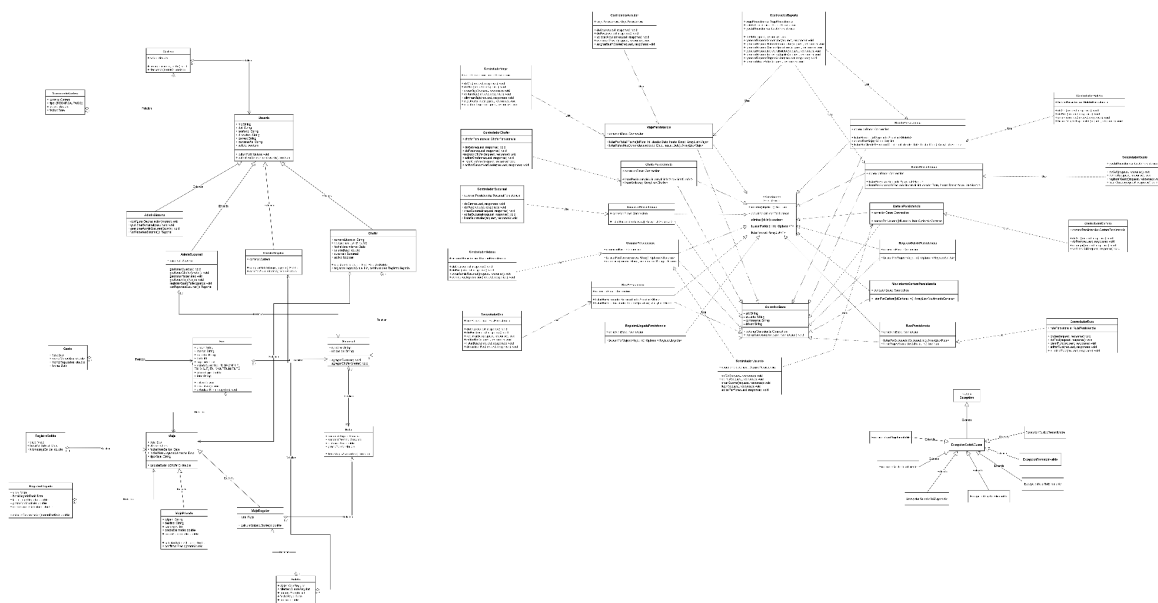
ControladorReporte (/reporte)

Gestiona la generación de los nueve reportes del sistema, aplicando los filtros

correspondientes según el rol del usuario (Administrador de Sucursal o Administrador del Sistema), y la exportación de dichos reportes en formato HTML.

Diagramas del Sistema

Diagrama de Clases

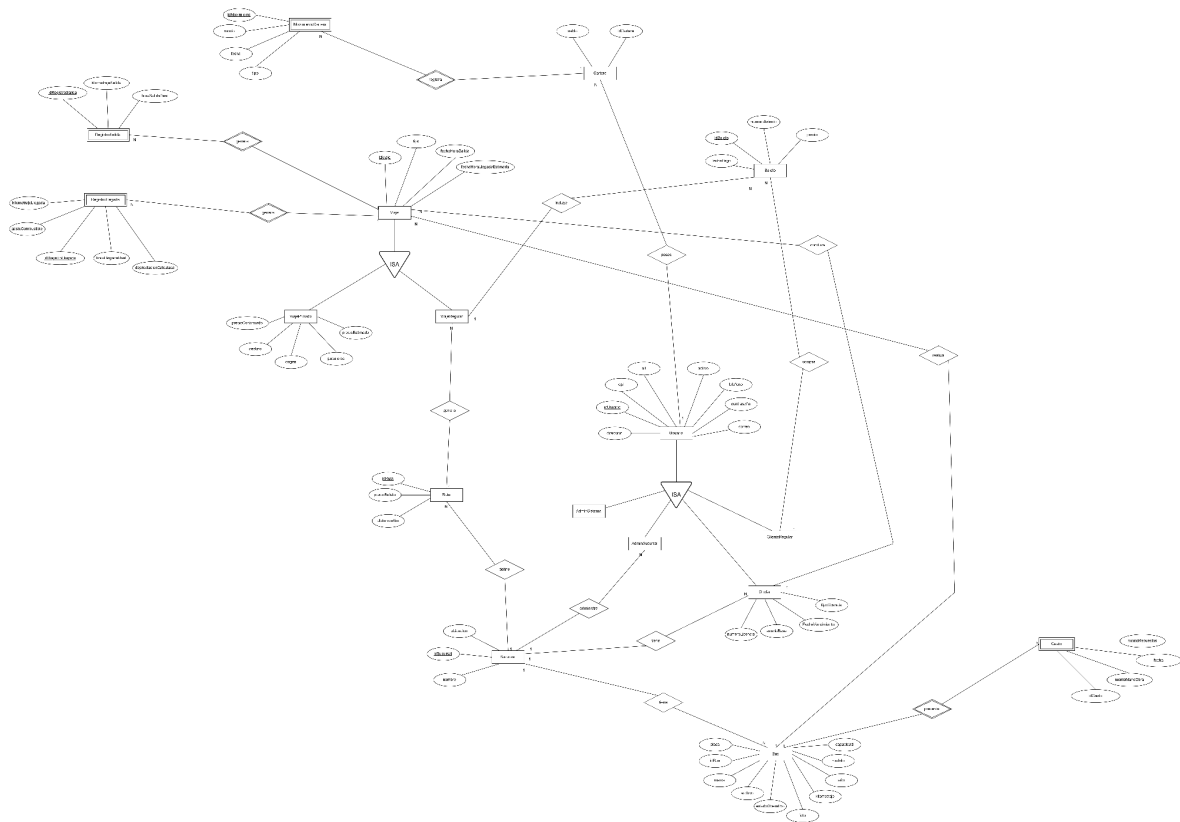


El diagrama de clases representa la estructura orientada a objetos del sistema, organizada en tres capas: entidades (modelo de negocio), persistencia (acceso a datos, incluyendo la interfaz genérica **Persistencia<T>**), y controladores (Servlets). Se destaca la jerarquía de herencia de **Usuario** hacia sus cuatro subclases, y de la clase abstracta **Viaje** hacia **ViajeRegular** y **ViajePrivado**.

Para visualizar el diagrama completo también se puede acceder al siguiente enlace:

<https://drive.google.com/file/d/1rtGr88UaEKXV2qzEDUuGrc1TVDkOJdlx/view?usp=sharing>

Diagrama Entidad-Relación

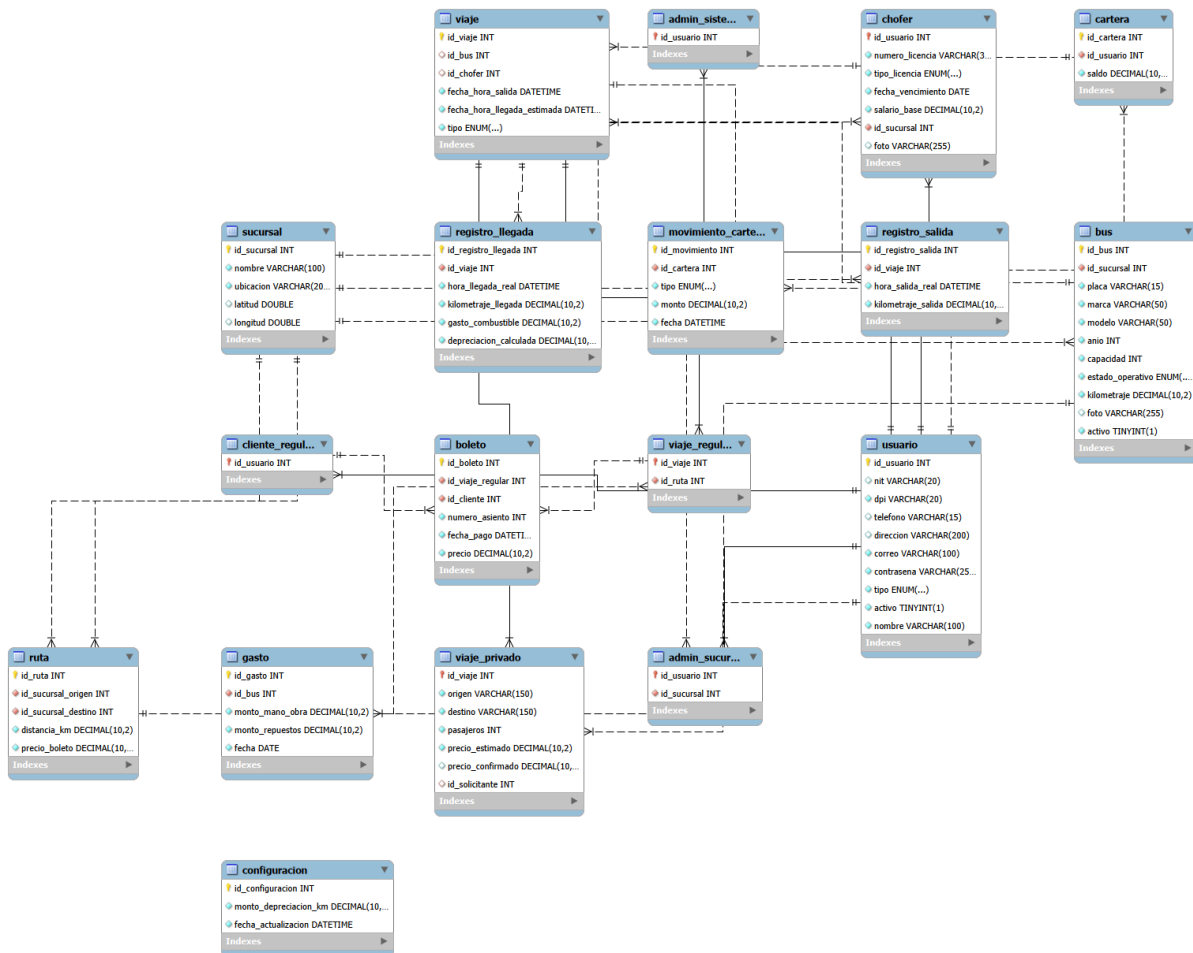


El diagrama entidad-relación, elaborado bajo la notación de Chen, representa el modelo de datos del sistema. Se utiliza la representación ISA (herencia) para reflejar la especialización de **Usuario** en sus cuatro tipos, y de **Viaje** en sus dos tipos. Las entidades débiles (**MovimientoCartera**, **RegistroSalida**, **RegistroLlegada**, **Gasto**) se representan con notación de rectángulo doble, al depender de una entidad fuerte para su existencia.

Para visualizar el diagrama completo también se puede acceder al siguiente enlace:

https://drive.google.com/file/d/1kH204Pf2AHxi7QeALVf_Y3mBJl6_akJa/view?usp=sharing

Diagrama de Tablas



El diagrama de tablas representa la estructura física final de la base de datos **gestion_buses**, generado a partir del script SQL implementado, incluyendo las claves primarias, claves foráneas, y las relaciones entre las diecisiete tablas que conforman el sistema.

Mapeo Físico de la Base de Datos

La estrategia de mapeo objeto-relacional utilizada para representar la herencia de **Usuario** y **Viaje** es la denominada "tabla por subclase" (*table per subclass*), en la cual cada clase padre cuenta con su propia tabla (**usuario**, **viaje**) que contiene los atributos comunes, y cada subclase cuenta con una tabla adicional (**admin_sistema**, **admin_sucursal**, **chofer**, **cliente_regular**; **viaje_regular**, **viaje_privado**) que contiene únicamente sus atributos específicos, relacionada con la tabla padre mediante una clave foránea que a su vez es su propia clave primaria.

Esta estrategia fue seleccionada frente a otras alternativas (como una única tabla con todos los atributos posibles, o una tabla independiente por cada clase sin relación entre sí) debido a que permite mantener la base de datos normalizada, evitando la presencia de columnas con valores nulos para los atributos que no apliquen a un tipo específico de usuario o viaje, a la vez que preserva la relación de herencia mediante claves foráneas compartidas.

```
-- =====
```

```
-- Base de datos: Gestion de Buses (IPC2 - Proyecto 1)
```

```
-- =====
```

```
CREATE DATABASE IF NOT EXISTS gestion_buses;
```

```
USE gestion_buses;
```

```
-- =====
```

```
-- SUCURSAL
```

```
-- =====
```

```
CREATE TABLE sucursal (
```

```
    id_sucursal INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nombre VARCHAR(100) NOT NULL,
```

```
    ubicacion VARCHAR(200) NOT NULL,
```

```
    latitud DOUBLE,
```

```
    longitud DOUBLE
```

```
);
```

```
-- =====
```

```
-- USUARIO (tabla base)
```

```
-- =====
```

```
CREATE TABLE usuario (
```

```
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
```

```

nombre VARCHAR(100) NOT NULL,
nit VARCHAR(20),
dpi VARCHAR(20) NOT NULL UNIQUE,
telefono VARCHAR(15),
direccion VARCHAR(200),
correo VARCHAR(100) NOT NULL UNIQUE,
contrasena VARCHAR(255) NOT NULL,
tipo ENUM('ADMIN_SISTEMA', 'ADMIN_SUCURSAL', 'CHOFER', 'CLIENTE') NOT
NULL,
activo BOOLEAN NOT NULL DEFAULT TRUE
);

```

```

-- =====

```

```

-- ADMIN_SISTEMA (hereda de usuario)

```

```

-- =====

```

```

CREATE TABLE admin_sistema (
    id_usuario INT PRIMARY KEY,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

```

```

-- =====

```

```

-- ADMIN_SUCURSAL (hereda de usuario)

```

```

-- =====

```

```

CREATE TABLE admin_sucursal (
    id_usuario INT PRIMARY KEY,
    id_sucursal INT NOT NULL,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),

```

```
FOREIGN KEY (id_sucursal) REFERENCES sucursal(id_sucursal)
);
```

```
-- =====
```

```
-- CHOFER (hereda de usuario)
```

```
-- =====
```

```
CREATE TABLE chofer (
    id_usuario INT PRIMARY KEY,
    numero_licencia VARCHAR(30) NOT NULL,
    tipo_licencia ENUM('A', 'B', 'C', 'M') NOT NULL,
    fecha_vencimiento DATE NOT NULL,
    salario_base DECIMAL(10,2) NOT NULL,
    id_sucursal INT NOT NULL,
    foto VARCHAR(255),
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario),
    FOREIGN KEY (id_sucursal) REFERENCES sucursal(id_sucursal)
);
```

```
-- =====
```

```
-- CLIENTE_REGULAR (hereda de usuario)
```

```
-- =====
```

```
CREATE TABLE cliente_regular (
    id_usuario INT PRIMARY KEY,
    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);
```

```

-- =====

-- BUS

-- =====

CREATE TABLE bus (

    id_bus INT AUTO_INCREMENT PRIMARY KEY,

    id_sucursal INT NOT NULL,

    placa VARCHAR(15) NOT NULL UNIQUE,

    marca VARCHAR(50) NOT NULL,

    modelo VARCHAR(50) NOT NULL,

    anio INT NOT NULL,

    capacidad INT NOT NULL,

    estado_operativo ENUM('DISPONIBLE', 'EN_VIAJE', 'EN_MANTENIMIENTO') NOT
    NULL DEFAULT 'DISPONIBLE',

    kilometraje DECIMAL(10,2) NOT NULL DEFAULT 0,

    foto VARCHAR(255),

    activo BOOLEAN NOT NULL DEFAULT TRUE,

    FOREIGN KEY (id_sucursal) REFERENCES sucursal(id_sucursal)

);

```

```

-- =====

-- RUTA

-- =====

CREATE TABLE ruta (

    id_ruta INT AUTO_INCREMENT PRIMARY KEY,

    id_sucursal_origen INT NOT NULL,

    id_sucursal_destino INT NOT NULL,

    distancia_km DECIMAL(10,2) NOT NULL,

```

```
precio_boleto DECIMAL(10,2) NOT NULL,  
  
FOREIGN KEY (id_sucursal_origen) REFERENCES sucursal(id_sucursal),  
  
FOREIGN KEY (id_sucursal_destino) REFERENCES sucursal(id_sucursal)  
);
```

```
-- =====
```

```
-- VIAJE (tabla base - estrategia tabla por subclase)
```

```
-- =====
```

```
CREATE TABLE viaje (  
  
    id_viaje INT AUTO_INCREMENT PRIMARY KEY,  
  
    id_bus INT NULL,  
  
    id_chofer INT NULL,  
  
    fecha_hora_salida DATETIME NOT NULL,  
  
    fecha_hora_llegada_estimada DATETIME NOT NULL,  
  
    tipo ENUM('REGULAR', 'PRIVADO') NOT NULL,  
  
    FOREIGN KEY (id_bus) REFERENCES bus(id_bus),  
  
    FOREIGN KEY (id_chofer) REFERENCES chofer(id_usuario)  
);
```

```
-- =====
```

```
-- VIAJE_REGULAR (hereda de viaje)
```

```
-- =====
```

```
CREATE TABLE viaje_regular (  
  
    id_viaje INT PRIMARY KEY,  
  
    id_ruta INT NOT NULL,  
  
    FOREIGN KEY (id_viaje) REFERENCES viaje(id_viaje),
```

```

        FOREIGN KEY (id_ruta) REFERENCES ruta(id_ruta)

);

-- =====

-- VIAJE_PRIVADO (hereda de viaje)

-- =====

CREATE TABLE viaje_privado (

    id_viaje INT PRIMARY KEY,

    origen VARCHAR(150) NOT NULL,

    destino VARCHAR(150) NOT NULL,

    pasajeros INT NOT NULL,

    precio_estimado DECIMAL(10,2) NOT NULL,

    precio_confirmado DECIMAL(10,2),

    id_solicitante INT,

    FOREIGN KEY (id_viaje) REFERENCES viaje(id_viaje),

    FOREIGN KEY (id_solicitante) REFERENCES usuario(id_usuario)

);

-- =====

-- BOLETO

-- =====

CREATE TABLE boleto (

    id_boleto INT AUTO_INCREMENT PRIMARY KEY,

    id_viaje_regular INT NOT NULL,

    id_cliente INT NOT NULL,

    numero_asiento INT NOT NULL,

```

```

    fecha_pago DATETIME NOT NULL,

    precio DECIMAL(10,2) NOT NULL,

    FOREIGN KEY (id_viaje_regular) REFERENCES viaje_regular(id_viaje),

    FOREIGN KEY (id_cliente) REFERENCES cliente_regular(id_usuario)

);

```

```

-- =====

```

```

-- CARTERA

```

```

-- =====

```

```

CREATE TABLE cartera (

    id_cartera INT AUTO_INCREMENT PRIMARY KEY,

    id_usuario INT NOT NULL UNIQUE,

    saldo DECIMAL(10,2) NOT NULL DEFAULT 0,

    FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)

);

```

```

-- =====

```

```

-- MOVIMIENTO_CARTERA

```

```

-- =====

```

```

CREATE TABLE movimiento_cartera (

    id_movimiento INT AUTO_INCREMENT PRIMARY KEY,

    id_cartera INT NOT NULL,

    tipo ENUM('RECARGA', 'PAGO') NOT NULL,

    monto DECIMAL(10,2) NOT NULL,

    fecha DATETIME NOT NULL,

    FOREIGN KEY (id_cartera) REFERENCES cartera(id_cartera)

```

);

-- =====

-- REGISTRO_SALIDA (inmutable)

-- =====

```
CREATE TABLE registro_salida (  
    id_registro_salida INT AUTO_INCREMENT PRIMARY KEY,  
    id_viaje INT NOT NULL UNIQUE,  
    hora_salida_real DATETIME NOT NULL,  
    kilometraje_salida DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (id_viaje) REFERENCES viaje(id_viaje)  
);
```

-- =====

-- REGISTRO_LLEGADA (inmutable)

-- =====

```
CREATE TABLE registro_llegada (  
    id_registro_llegada INT AUTO_INCREMENT PRIMARY KEY,  
    id_viaje INT NOT NULL UNIQUE,  
    hora_llegada_real DATETIME NOT NULL,  
    kilometraje_llegada DECIMAL(10,2) NOT NULL,  
    gasto_combustible DECIMAL(10,2) NOT NULL,  
    depreciacion_calculada DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (id_viaje) REFERENCES viaje(id_viaje)  
);
```

```
-- =====  
-- GASTO (taller y repuestos)
```

```
-- =====
```

```
CREATE TABLE gasto (  
    id_gasto INT AUTO_INCREMENT PRIMARY KEY,  
    id_bus INT NOT NULL,  
    monto_mano_obra DECIMAL(10,2) NOT NULL,  
    monto_repuestos DECIMAL(10,2) NOT NULL,  
    fecha DATE NOT NULL,  
    FOREIGN KEY (id_bus) REFERENCES bus(id_bus)  
);
```

```
-- =====
```

```
-- CONFIGURACION (para el monto de depreciacion por km)
```

```
-- =====
```

```
CREATE TABLE configuracion (  
    id_configuracion INT AUTO_INCREMENT PRIMARY KEY,  
    monto_depreciacion_km DECIMAL(10,4) NOT NULL,  
    fecha_actualizacion DATETIME NOT NULL  
);
```

Pseudocódigo de los Métodos más Importantes

1. Autenticación de usuario (**UsuarioPersistencia.autenticar()**)

FUNCION autenticar(correo, contraseña)

ejecutar consulta: buscar tipo de usuario donde correo y contraseña coincidan y este activo

SI se encontró un resultado ENTONCES

obtener el tipo de usuario (ADMIN_SISTEMA, ADMIN_SUCURSAL, CHOFER, CLIENTE)

SEGUN el tipo:

CASO "ADMIN_SISTEMA": retornar
AdminSistemaPersistencia.buscarPorCorreo(correo)

CASO "ADMIN_SUCURSAL": retornar
AdminSucursalPersistencia.buscarPorCorreo(correo)

CASO "CHOFER": retornar ChoferPersistencia.buscarPorCorreo(correo)

CASO "CLIENTE": retornar ClienteRegularPersistencia.buscarPorCorreo(correo)

FIN SEGUN

SINO

retornar vacío

FIN SI

FIN FUNCION

2. Cálculo de depreciación al registrar llegada (**RegistroLlegada**, constructor)

FUNCION calcularDepreciacion(kilometrajeSalida, kilometrajeLlegada, montoPorKm)

kilometrosRecorridos = kilometrajeLlegada - kilometrajeSalida

depreciacion = kilometrosRecorridos * montoPorKm

retornar depreciacion

FIN FUNCION

3. Validación de disponibilidad de horario

(**ViajeRegularPersistencia.choferTieneViajeEnHorario()**)

FUNCION choferTieneViajeEnHorario(idChofer, fechaSalida, fechaLlegada, idViajeExcluir)

consulta = "contar viajes donde id_chofer = idChofer

Y fecha_salida < fechaLlegada

Y fecha_llegada > fechaSalida"

SI idViajeExcluir no es nulo ENTONCES

agregar condicion "Y id_viaje != idViajeExcluir" a la consulta

FIN SI

ejecutar consulta

retornar (total de resultados > 0)

FIN FUNCION

4. Compra de boleto (**ControladorBoleto.comprarBoleto()**)

FUNCION comprarBoleto(idViaje, numeroAsiento, cliente)

 obtener viaje por idViaje

 SI numeroAsiento fuera del rango de capacidad del bus ENTONCES

 lanzar ExcepcionFormatoInvalido

 FIN SI

 SI el asiento ya esta en la lista de asientos ocupados ENTONCES

 lanzar ExcepcionAsientoNoDisponible

 FIN SI

 obtener cartera del cliente

 SI saldo de cartera es menor al precio del boleto ENTONCES

 lanzar ExcepcionCreditoInsuficiente

 FIN SI

 INICIAR TRANSACCION

 registrar el boleto

 descontar el saldo de la cartera

 registrar el movimiento de tipo PAGO

 CONFIRMAR TRANSACCION

FIN FUNCION

5. Confirmación de precio y asignación de alquiler (ControladorAlquiler.confirmarPrecio())

FUNCION confirmarPrecio(idViaje, precioConfirmado, idBus, idChofer)

obtener el viaje privado por idViaje

SI choferTieneViajeEnHorario(idChofer, fechas del viaje) ENTONCES

mostrar error "chofer no disponible en ese horario"

retornar

FIN SI

SI busTieneViajeEnHorario(idBus, fechas del viaje) ENTONCES

mostrar error "bus no disponible en ese horario"

retornar

FIN SI

actualizar precio confirmado del viaje

asignar bus y chofer al viaje

guardar cambios

FIN FUNCION

Diagrama de Flujo

El diagrama de flujo representa el recorrido general que sigue un usuario dentro del sistema CodeNBuses, desde el primer acceso a la aplicación hasta la realización de una acción específica según su rol. Tras autenticarse exitosamente, el sistema identifica el tipo de usuario y presenta el menú de opciones correspondiente a sus privilegios (Administrador del Sistema, Administrador de Sucursal, Chofer, o Cliente Regular). Cada acción realizada por el usuario pasa por un proceso de validación de datos y reglas de negocio antes de ser registrada en la base de datos, garantizando la integridad de la información del sistema en todo momento.

[INICIO]



[Usuario accede a la aplicación]



[Página de Bienvenida]



¿Tiene cuenta? —NO—→ [Crear Cuenta] —→ [Login]



[Login: ingresa correo/contraseña]



¿Credenciales correctas? —NO—→ [Mostrar error] —→ (vuelve a Login)



[Sistema identifica el tipo de usuario]



¿Qué rol es?

|— AdminSistema —→ [Menú: Sucursales, Admins, Reportes, Config. Depreciación]

|— AdminSucursal —→ [Menú: Buses, Choferes, Rutas, Viajes, Gastos, Alquileres, Reportes]

|—— Chofer ——→ [Menú: Mis Viajes, Alquiler Privado]

└—— ClienteRegular ——→ [Menú: Comprar Boleto, Alquiler Privado, Mi Cartera]

↓

[Usuario realiza la acción correspondiente]

↓

[Sistema valida datos y reglas de negocio]

↓

¿Validación exitosa? —NO——→ [Mostrar mensaje de error] ——→ (vuelve al formulario)

↓ SÍ

[Sistema registra/actualiza información en la base de datos]

↓

[Sistema muestra confirmación al usuario]

↓

[FIN o vuelve al menú principal]

Conclusión

El desarrollo del sistema CodeNBuses permitió aplicar de manera integral los conocimientos adquiridos en el curso de Introducción a la Programación y Computación 2, abarcando desde el análisis de requerimientos mediante la técnica de casos de uso, hasta la implementación completa de una aplicación web dinámica utilizando la arquitectura cliente-servidor con JavaServer Pages.

La separación del sistema bajo el patrón de diseño Modelo-Vista-Controlador permitió mantener una arquitectura ordenada y escalable, facilitando la incorporación progresiva de los distintos módulos del sistema (gestión de sucursales, buses, choferes, rutas, viajes, boletos, cartera digital, alquileres privados, gastos de taller, y reportes financieros) sin comprometer la estabilidad de las funcionalidades ya implementadas.

Asimismo, la aplicación de conceptos de programación orientada a objetos, como la herencia y el polimorfismo en la jerarquía de usuarios y viajes, permitió modelar de forma natural la relación existente entre las distintas entidades del negocio, reduciendo la duplicación de código y facilitando el mantenimiento del sistema. El uso de transacciones en las operaciones que involucran múltiples tablas, así como el manejo de excepciones personalizadas, contribuyó a garantizar la integridad y consistencia de la información almacenada en la base de datos, incluso ante escenarios de error o datos inválidos.

Finalmente, el proceso de pruebas exhaustivas realizado a lo largo del desarrollo permitió identificar y corregir diversas inconsistencias en el manejo de fechas, disponibilidad de recursos, y validaciones de negocio, fortaleciendo la robustez final del sistema y asegurando que su comportamiento responda de manera fiel a los requerimientos establecidos por el cliente en la descripción del problema.